

YallaFix Database Schema

Entity-Relationship Diagram & Table Definitions

Version 1.0 | May 2026

1. Overview

YallaFix uses PostgreSQL 13+ hosted on Supabase. The database consists of 9 main tables designed to support multi-role facility management with complete audit trails and notification tracking.

1.1 Connection Details

- Host: Supabase (aws-0-eu-west-1.pooler.supabase.com)
- Port: 6543 (using connection pooler)
- Database: postgres
- Username: postgres

1.2 Key Constraints

- Foreign Key Cascading: ON DELETE CASCADE for data integrity
- UNIQUE constraints on email and category names
- CHECK constraints on status and role enumerations

2. Entity-Relationship Diagram (ERD)

Below is the complete Entity-Relationship Diagram for the YallaFix system. It shows all tables, their columns, and the relationships between them.

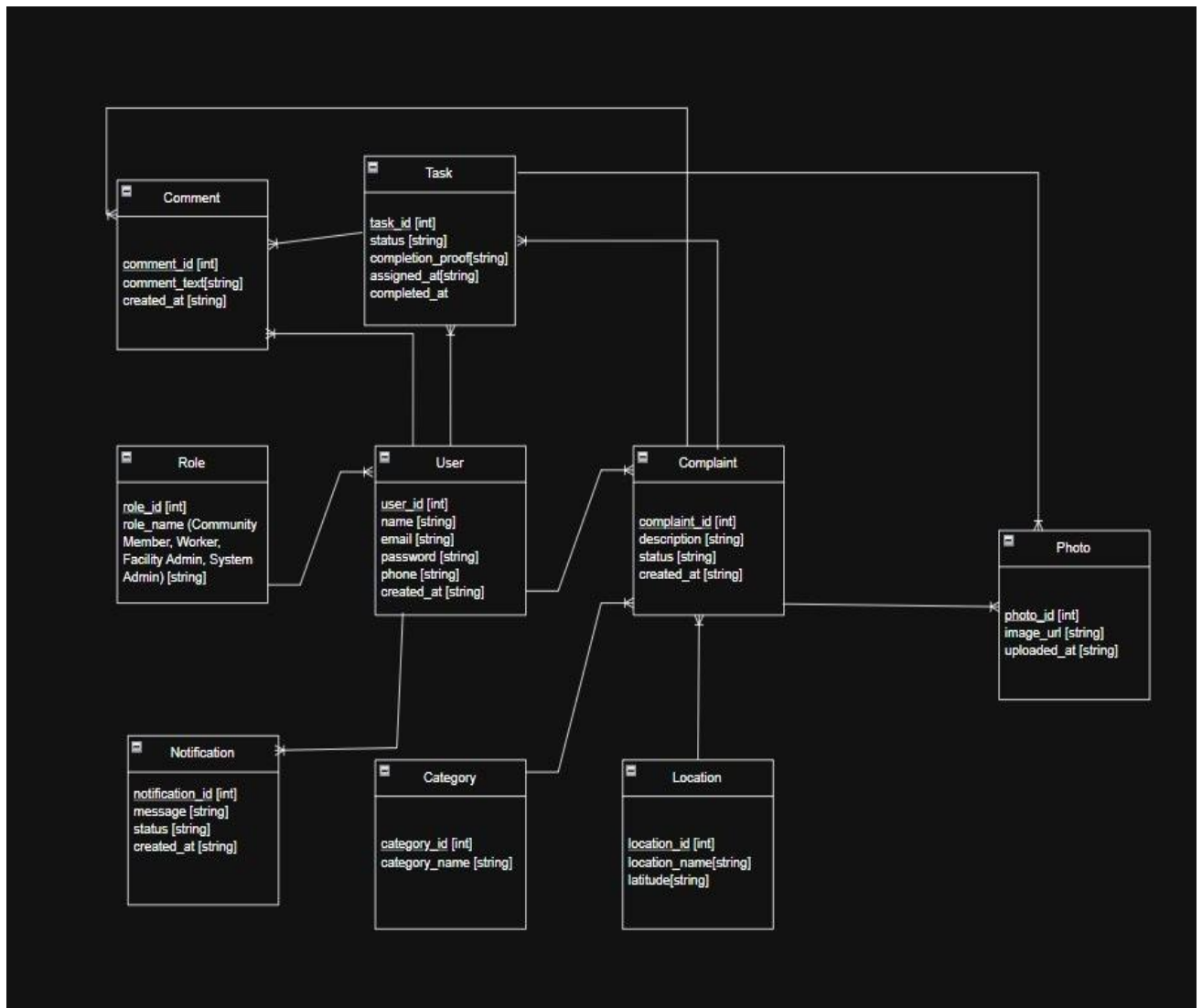


Figure 2.1: YallaFix Database Entity-Relationship Diagram

2.1 Diagram Components

Tables (Entities)

- Users: Stores all users with roles and authentication details
- Roles: Defines different user roles (Community Member, Worker, Facility Admin, System Admin)
- Complaints: Core entity tracking facility issues
- Tasks: Assignments of work to workers (also called 'assignments')
- Comments: User discussions and notes on complaints
- Photos: Issue documentation images
- Categories: Issue classification (Electrical, Plumbing, etc.)
- Locations: Campus building and location data

- Notifications: Real-time alerts for users

Key Relationships

- User → Role: Many-to-One (User belongs to one Role)
- User → Complaint: One-to-Many (User submits multiple Complaints)
- Complaint → Category: Many-to-One (Complaint belongs to one Category)
- Complaint → Location: Many-to-One (Complaint is at one Location)
- Complaint → Photo: One-to-Many (Complaint has multiple Photos)
- Complaint → Comment: One-to-Many (Complaint has multiple Comments)
- Complaint → Task: One-to-One (Complaint has one Task/Assignment)
- Task → User: Many-to-One (Task assigned to one Worker/User)
- User → Notification: One-to-Many (User receives multiple Notifications)

3. Table Definitions

3.1 users

Stores all registered users across all roles with authentication credentials.

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-incrementing user ID
name	TEXT	NOT NULL	User's full name
email	TEXT	UNIQUE, NOT NULL	Email address
password	VARCHAR(255)	NOT NULL	Bcrypt hashed password
role_id	INTEGER	FOREIGN KEY	References roles table
created_at	TIMESTAMP	DEFAULT NOW()	Account creation time

3.2 categories

Predefined issue categories managed by administrators.

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Category ID
name	TEXT	UNIQUE, NOT NULL	Category name
description	TEXT		Category description

3.3 locations

Campus locations where complaints can be reported.

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Location ID

name	TEXT	NOT NULL	Location name
building	TEXT		Building identifier
floor	TEXT		Floor number/name
room	TEXT		Room number/name

3.4 complaints

Core table storing all facility issues submitted by community members.

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Complaint ID
reporter_id	INTEGER	FK to users	Who reported it
category_id	INTEGER	FK to categories	Issue category
location_id	INTEGER	FK to locations	Where it is
description	TEXT	NOT NULL	Issue description
image_url	TEXT		Photo URL
status	TEXT	CHECK	open/in_progress/resolved/closed
created_at	TIMESTAMP	DEFAULT NOW()	Submit time

3.5 comments

Stores discussion threads and official updates on complaints.

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Comment ID
complaint_id	INTEGER	FK to complaints	Which complaint
user_id	INTEGER	FK to users	Comment author
text	TEXT	NOT NULL	Comment content
created_at	TIMESTAMP	DEFAULT NOW()	Comment time

3.6 notifications

Real-time alerts sent to users on status changes and assignments.

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Notification ID
user_id	INTEGER	FK to users	For which user

complaint_id	INTEGER	FK to complaints	Related complaint
message	TEXT	NOT NULL	Notification text
status	TEXT		Read/unread status
created_at	TIMESTAMP	DEFAULT NOW()	When sent

4. Recommended Indexes

- CREATE INDEX idx_complaints_status ON complaints(status)
- CREATE INDEX idx_complaints_reporter_id ON complaints(reporter_id)
- CREATE INDEX idx_notifications_user_id ON notifications(user_id)
- CREATE INDEX idx_notifications_is_read ON notifications(is_read)
- CREATE INDEX idx_assignments_worker_id ON assignments(worker_id)
- CREATE INDEX idx_users_email ON users(email)